CS & IT ENGINEERING



Algorithms

Analysis of Algorithms



Lecture No.- 3 ||

By- Aditya sir

Recap of Previous Lecture



Topic

Topic

TC analysis of Recursive Algo.

Topics to be Covered



- Topic
- Topic
- Topic

Practice Questions

Loop Complexities



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.

Telegram



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW



Topic : Analysis of Algorithms

#Q. Given a recurrance, determine the time compleixty.

$T(n) = 2 , n = 2$

$T(n) = \sqrt{n} * (\sqrt{n}) + n, n > 2$

A $O(nlogn)$

B $O(nlog\ (logn))$

C $O(n^2)$

D $O(log\ (logn))$



Topic : Analysis of Algorithms

```
Algo AJ( n)
{
    if (n == 2)
        return 1
    else
        return [AJ(√n) + 10]
}
```

T(n)

H(W)

b

T(√n)

TC : O(log log n)

Step 1:- $T(n) = b, n=2$

$T(n) = T(\sqrt{n}) + a, n>2$

$$T(n) = T(\sqrt{n}) + a$$

$$T(n) = T(n^{1/2}) + a \quad \text{——} (1)$$

$$T(n^{1/2}) = T(n^{\frac{1}{2}\times\frac{1}{2}}) + a$$

$$T(n) = T(n^{1/2^2}) + a + a$$

$$T(n) = T(n^{1/2^2}) + 2a \quad \text{——} (2)$$

$$T(n^{1/2}) = T(n^{1/2^2}) + a$$

$$T(n) = T(n^{1/2^3}) + a + 2a$$

$$T(n) = T(n^{1/2^3}) + 3a \quad \text{---} (3)$$

general term is,

$$T(n) = T(n^{1/2^k}) + k * a \quad \text{---} (4)$$

Base Condition,

$$n^{1/2^k} = 2$$

$$\frac{1}{2^k} \log n = \log_2 2 = 1$$

$$\log n = 2^k$$

$$k = \log(\log n)$$

$$T(n) = T(n^{1/2^k}) + k * a$$

$k = \log\log n$

$2^k = \log n$

$$T(n) = T(2) + \log(\log n) * a$$

$$T(n) = b + a * \log(\log n)$$ → Value.

Step 3 :- TC of AJ(n) is $O(\log(\log n))$



Topic : Analysis of Algorithms

86%

```
Algo AJ( n)
{
        if (n ==1)
            return n
        else
            return (AJ (n/2) + 10)
}
```

T(n)

b

T(n/2)

A) O(log n)

B) O(√n)

C) O(n²)

D) O(n)

Step 1 ; $T(n) = b, \underline{n=1}$

$T(n) = T(n/2) + a, \quad n > 1.$

↳ Binary Search

Step 2 :-

$$T(n) = T(n/2) + a \quad \text{---} (1)$$

$$T(n/2) = T(n/2^2) + a$$

$$T(n) = T(n/2^2) + 2a \quad \text{---} (2)$$

$$T(n) = T(n/2^3) + 3a \quad \text{---} (3)$$

General term

$$T(n) = T(n/2^k) + k * a \quad \text{---} (4)$$

for base Condition,

$$n/2^k = 1 \qquad a^k = n \qquad k = (\log_2 n)$$

$$T(n) = T(1) + a * \log n$$

$$\boxed{T(n) = b + a * \log_2 n} \longrightarrow \text{value}$$

$$TC = O(\log_2 n)$$

37.25 %

Algo AJ(n) → T(n)

```
{
    if(n==1)
        return 1          } b
    else
        return ( AJ(n/2) + AJ2(n) )
                 T(n/2)
}
```

A) $O(\sqrt{n})$
~~B) $O(n)$~~
C) $O(\log n)$
D) $O(n \log n)$

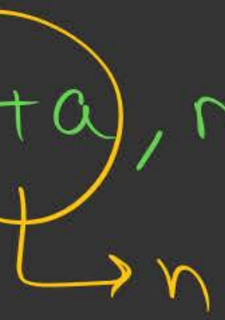
$O(n)$

Step 1 :

$$T(n) = b, \quad n = 1$$

$$T(n) = T(n/2) + O(n) + a, \quad n > 1$$

$$= T(n/2) + n + a, \quad n > 1$$



$$T(n) = T(n/2) + n, \quad n > 1$$

H.W

Tony Gaddis 2025

HW

$$T(n) = T(n/2) + n$$

$$T(n/2) = T(n/2^2) + n/2$$

$$T(n/2^2) = T(n/2^3) + n/2^2$$

$$T(n) = T(n/2^2) + n + n/2$$

$$T(n) = T(n/2^3) + \frac{n}{2^0} + n/2^1 + n/2^2$$

In general

$$T(n) = T(n/2^k) + n\left[\frac{1}{2^0} + \frac{1}{2^1} \cdots + \frac{1}{2^{k-1}}\right]$$

GP = $a = 1$

$r = 1/2$

$n = k$

$$S_n = 1\,\frac{\left(1 - 1/2^k\right)}{1 - 1/2} = 2\left(1 - 1/2^k\right)$$

$$T(n) = T(n/2^k) + 2n\left(1 - 1/2^k\right)$$

for B.C, $n/2^k = 1 \longrightarrow \boxed{2^k = n}$

$$T(n) = T(1) + 2n\left(1 - 1/n\right)$$

$$= b + 2n - \frac{2n}{n}$$

$$\boxed{T(n) = b + 2n - 2}$$

TC of $AJ(n) \longrightarrow O(n)$

Loop Complexities !

ini

while (condition)

{ updation

}

for(i=0; i<n; i++)

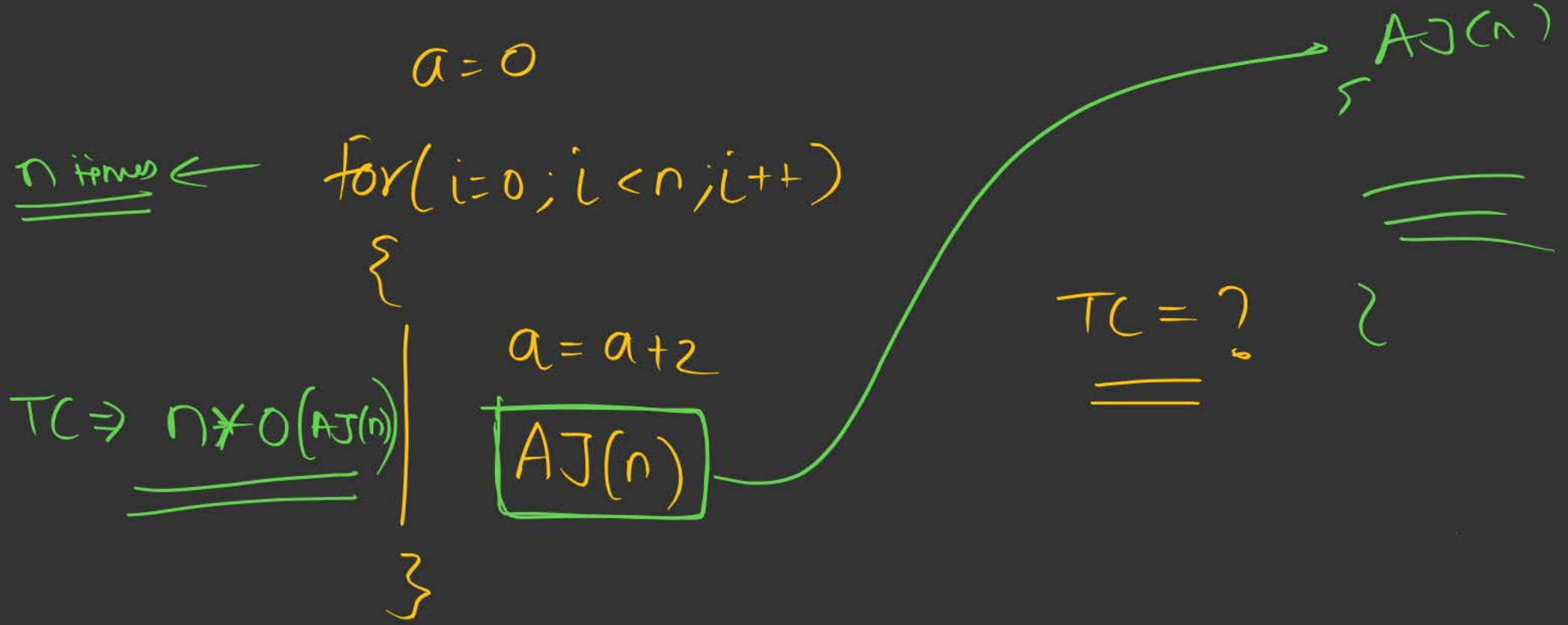
{

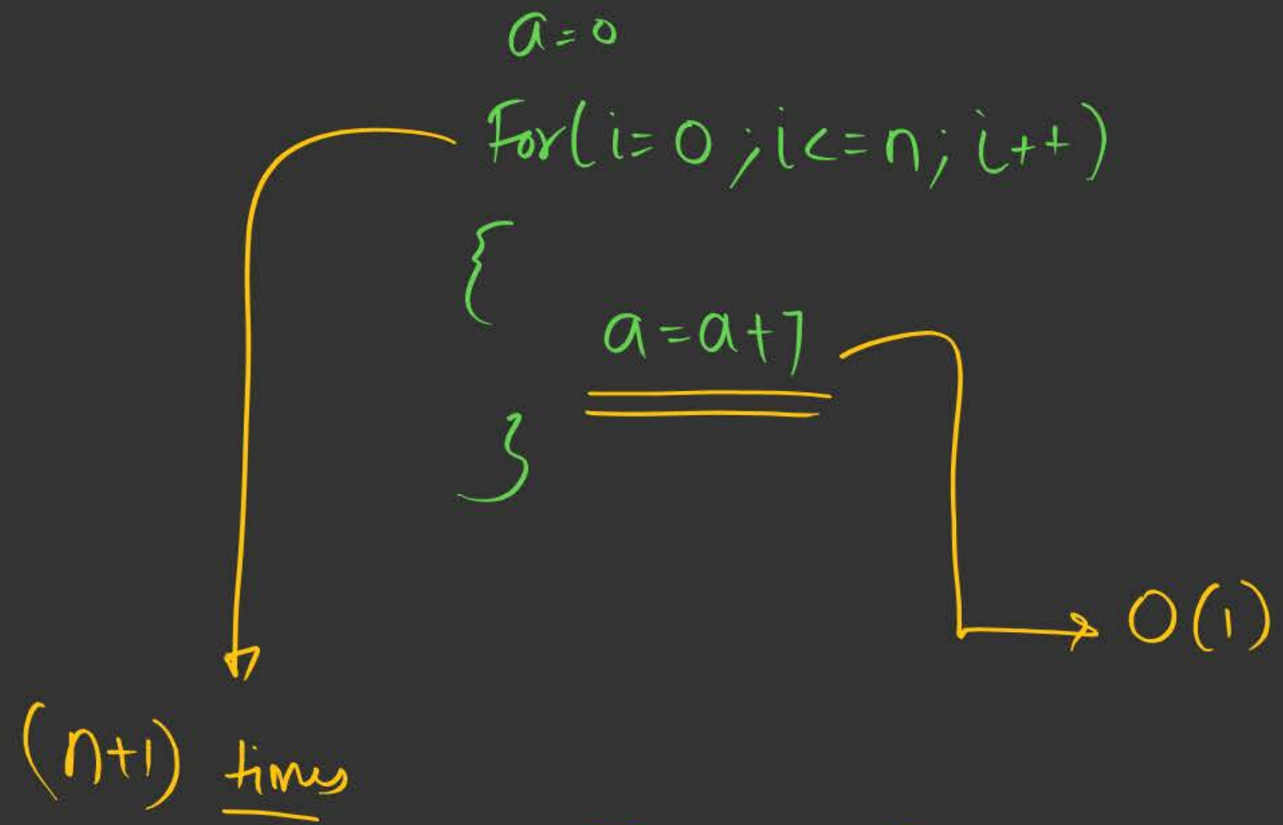
}

Loop Complexity :-

1) Runs ?

2) TC of loop statements / body


a = 0
n times
for(i=0 ; i<n ; i++)
{
a = a+2
AJ(n)
}
TC ⇒ n * O(AJ(n))
AJ(n)
{
TC = ?
}



$TC: (n+1) * O(1)$

$= O(n)$

$i=0 \longrightarrow O(1)$

$i=1 \longrightarrow O(1)$

$i=2 \longrightarrow O(1)$

$i=n \longrightarrow O(1)$

```
a = 0
for (i = n/2 ; i <= n ; i++)
{
    a = a + 10
}
```

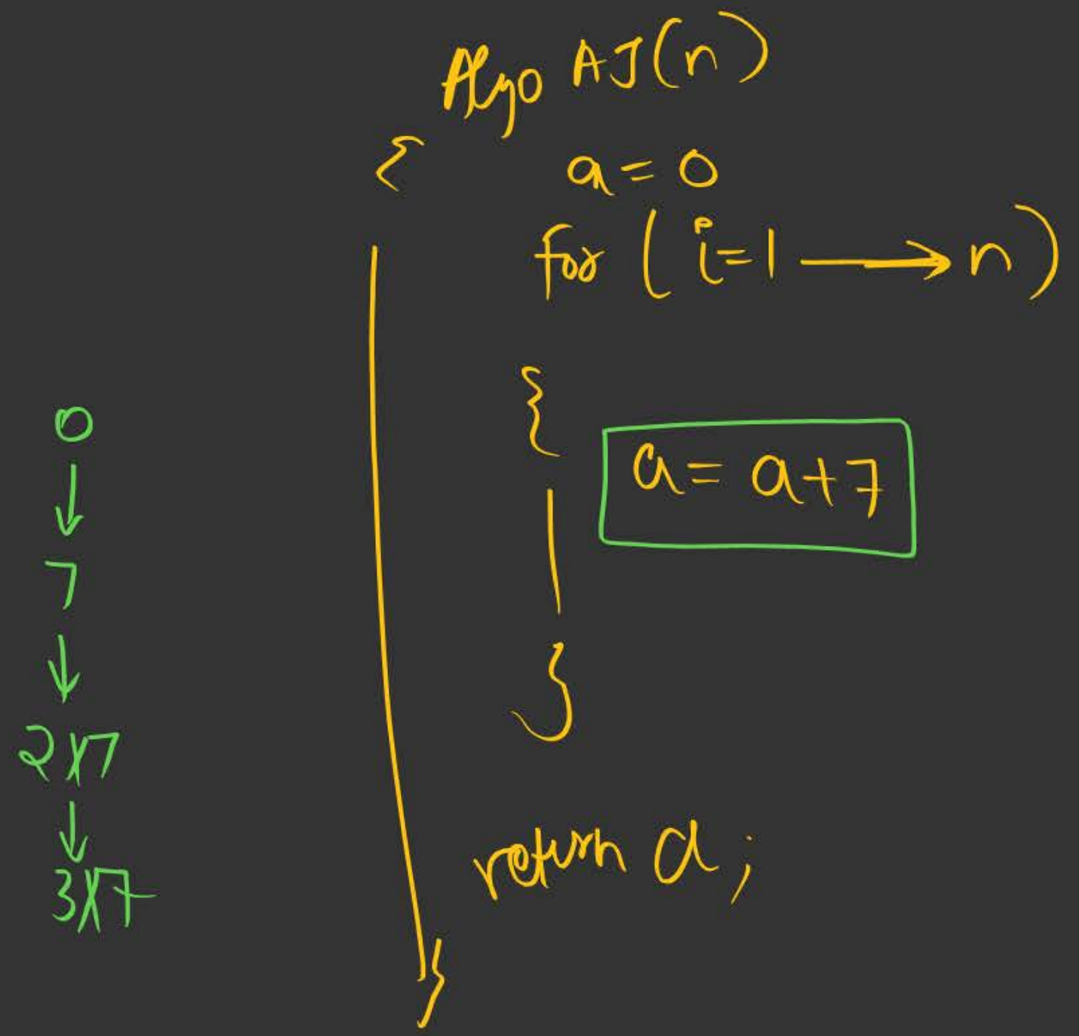
$a = a + 10$ → $O(1)$

$n/2 \longrightarrow n$

$n/2$ times.

TC: $O(1) * n/2$

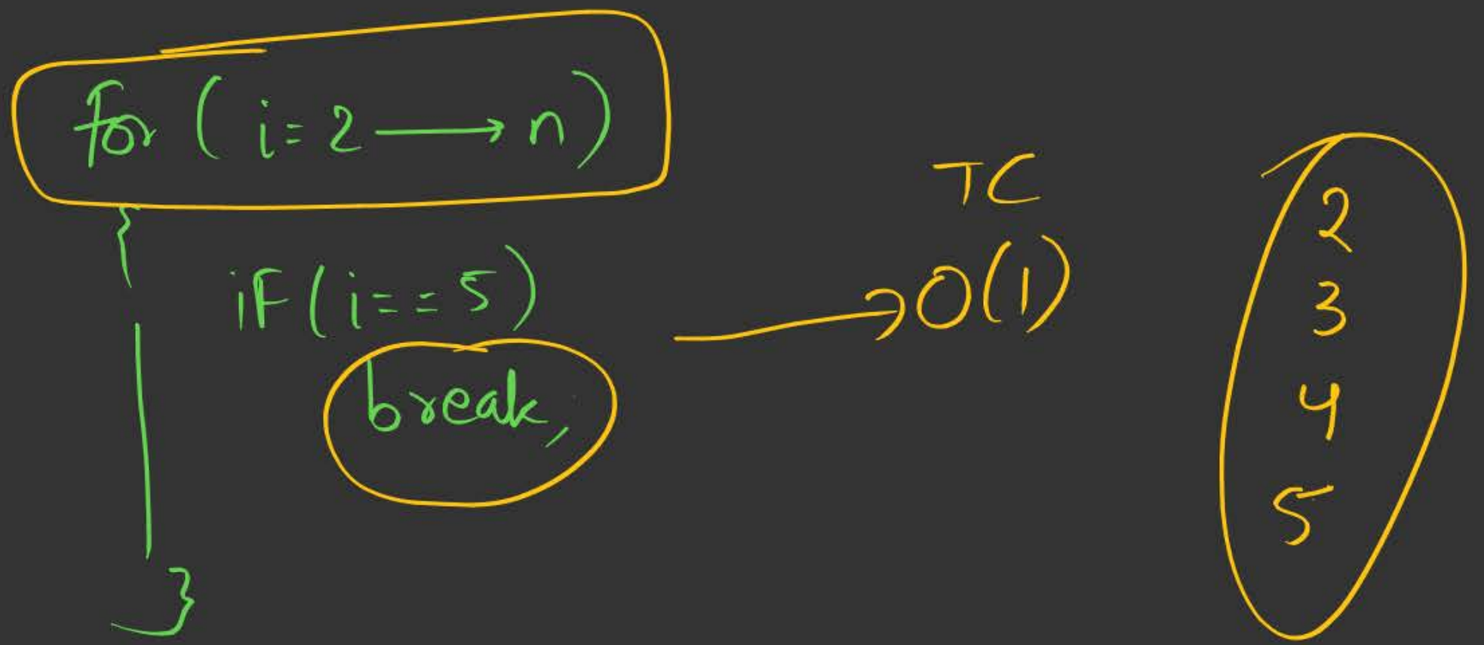
$= O(n/2) = O(n)$



1) TC of code? ⟶ O(n)

2) Value returned by AJ(n) is ____?

7 * n


for (i = 2 ⟶ n)
{
IF (i == 5)
break ;
}
TC
O(1)
2
3
4
5

```
a = 0
for( j = n ; j >= 0 ; j-- )
{
    a = a + j
}
print(a)
```

1) TC ?

2) O/p = ? → 'n'

$n \longrightarrow 0$

↳ n+1 runs

$T(n) \rightarrow O(n)$

$a = \sum_{i=0}^{n} i = \frac{n(n+1)}{2}$

a = 0 , j = n

a = 0 + n = n , j = n-1

a = n + (n-1)

n + (n-1) + (n-2) + ... 1 + 0

$n, x \rightarrow \geq 1$ (int)

```
Algo AJ(n, x)
{
    for(i=1; i<=n; i++)
    {
        if(x % i == 0)
            break
        print(i)
    }
}
```

$x \% 1 == 0$

1) B.C
2) W.C

} i/p



BC $\rightarrow O(1)$

WC $\rightarrow O(1)$

n, x -> > 1 (int)

```
Algo AJ(n, x)
{
    for(i=2; i<=n; i++)
    {
        if(x % i == 0)
            break
    }
    print(i)
}
```

x % 1 == 0

n = 10

x = 13

[1 → 10]

WC: O(n)

1) B.C
2) W.C
} i/p

a → prime > n

WC → O(n)

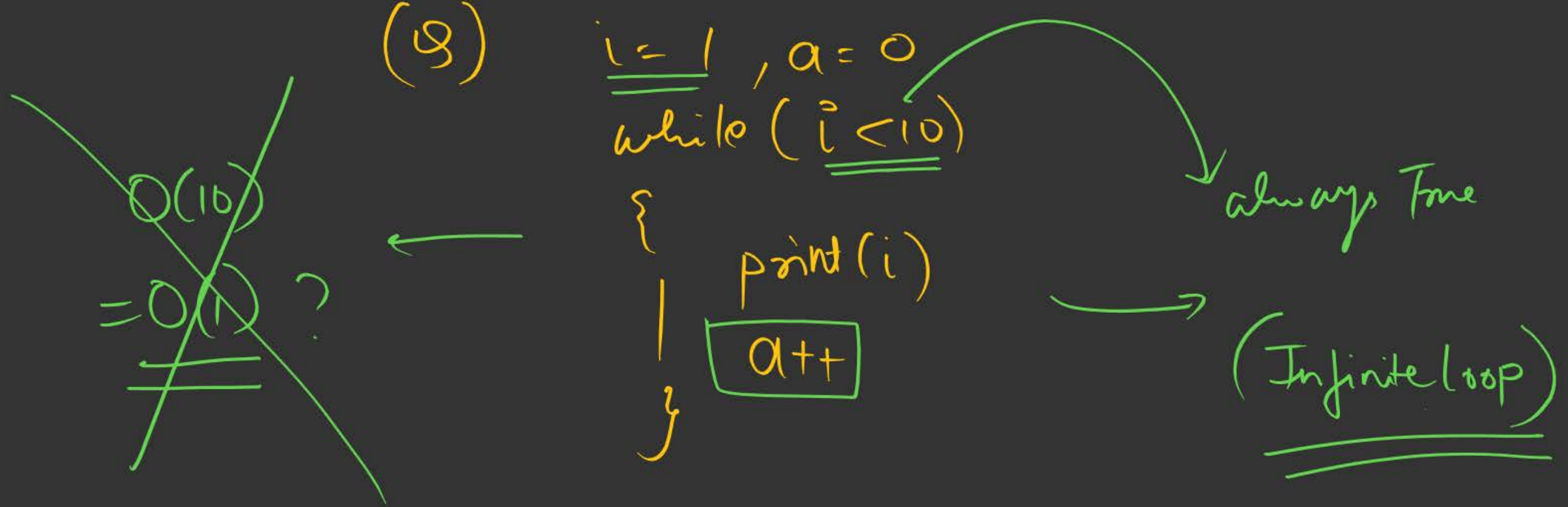
B.C, a = 2

↳ O(1)

(Q)

```
i = 1
while (i < n)
{
    i++
    print(i)
}
```

O(n) ←


(8)
i = 1 , a = 0
while (i <10)
{
print (i)
a++
}
always True
(Infinite loop)
O(10)
=O(1) ?

Q5 :

Consider the C program

```
int i, a=0
for( i=1; i<=n; i++) ;
a = a+i
print(a)
```

$1 \longrightarrow n$

$i = n+1$

$a = 0 + (n+1)$

$= (n+1)$

$i = n+1$

A) TC ? $O(n)$

B) O/P ? $(n+1)$

return AJ(n)+n

for + while

```
j=1
while(j<=n/2)
{
    print(n*n)
    j=j+1
}
```

$1 \xrightarrow[+1]{} n/2$

$\longrightarrow O(n/2) = \underline{\underline{O(n)}}$

```
j = n/2
while( j >= 1)
{
    print(n)
    j = j - 1
}
```

$1 \longrightarrow n/2$ (+1)

$n/2 \longrightarrow 1$ (-1)

$TC = O(n)$

```
j=1
while(j>=5)
{
    print(j)
    j=j+1
}
```

$\longrightarrow TC = O(1)$

Infinite

```
j=10
while(j>=5)
{
    print(j)
    j=j+1
}
```

→ always True

for + while

```
a = 0
for (i = 1 ⟶ n)  ⟶ n
{
    j = 1
    while (j ≤ n/2)
    {
        a = a + 3
    } j = j + 1
}
```

→ value of a after code ends.

$n/2$

$i = 1 \rightarrow O(n/2)$

$i = 2 \rightarrow O(n/2)$

$i = n \rightarrow O(n/2)$

$3 \times n \times n/2$ times

$\frac{3n^2}{2}$

nested

overall:

$n * O(n/2)$

$= O(n^2)$

42%

Imp

Challenge :-

$$T(n) = 2T(n/2) + n\log n, \quad n > 1$$

$$T(n) = a, \quad n = 1$$

A) $O(n^2 \log n)$

B) $O(n^2)$

C) $O(n(\log n)^2)$

D) $O(n \log n)$

$$T(n) = 2T(n/2) + n\log n$$

$$T(n/2) = 2T(n/2^2) + n/2 \log(n/2)$$

$$T(n) = 2\left[2T(n/2^2) + n/2 \log(n/2)\right] + n\log n$$

$$= 2^2 T(n/2^2) + n\log(n/2) + n\log n$$

$$T(n/2^2) = 2T(n/2^3) + n/2^2 \log(n/2^2)$$

$$T(n) = 2^2 \times 2T(n/2^3) + \cancel{2^2} \times n/\cancel{2^2} \log(n/2^2) + n\log(n/2) + n\log n$$

$$T(n) = 2^3 T(n/2^3) + n\left[\log(n) + \log(n/2) + \log(n/2^2)\right]$$

General condition.

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + n\left[\log\left(\frac{n}{2^0}\right) + \log\left(\frac{n}{2^1}\right) + \cdots + \log\left(\frac{n}{2^{k-1}}\right)\right]$$

$\log(ab) = \log a + \log b$

$$= 2^k T\left(\frac{n}{2^k}\right) + n\left[\log\left(\frac{n}{2^0} \times \frac{n}{2^1} \times \frac{n}{2^2} \cdots \times \frac{n}{2^{k-1}}\right)\right]$$

$$a^m \times a^n = a^{(m+n)}$$

$$\log\left(n/2^0 \times n/2^1 \cdots n/2^{k-1}\right)$$

$$\log\left(\frac{n^k}{2^0 \times 2^1 \times 2^2 \cdots 2^{k-1}}\right)$$

$$= \log(n^k) - \log\left(2^{(0+1+\cdots k-1)}\right)$$

$$= \left[\log_2(n^k) - \log_2\left(2^{\frac{k(k-1)}{2}}\right)\right]$$

$$= \left[k * \log n - \frac{k(k-1)}{2}\right]$$

$$0 \longrightarrow (k-1)$$

$$\Rightarrow \frac{(k-1)k}{2}$$

$$T(n) = 2^k T(n/2^k) + n\left[k \times \log n - \frac{k(k-1)}{2}\right]$$

$$n/2^k = 1 \quad 2^k = n \quad k = \log n$$

$$= n \times T(1) + n\left[\log n \times \log n - \frac{\log n(\log n - 1)}{2}\right]$$

$$= n \times b + n(\log n)^2 - \frac{n(\log n)}{2} + \frac{n \log n}{2}$$

$$TC = O(n(\log n)^2) = \left[b \times n + \frac{n(\log n)^2}{2} + \frac{n \log n}{2}\right]$$



Summary

→ TC Recurrence

→ loop Complexities



THANK - YOU